Understanding Hopfield Networks Through Hands-On Implementation

A minimalist object-oriented approach to associative memory in Python.

Hernández Pérez Erick Fernando

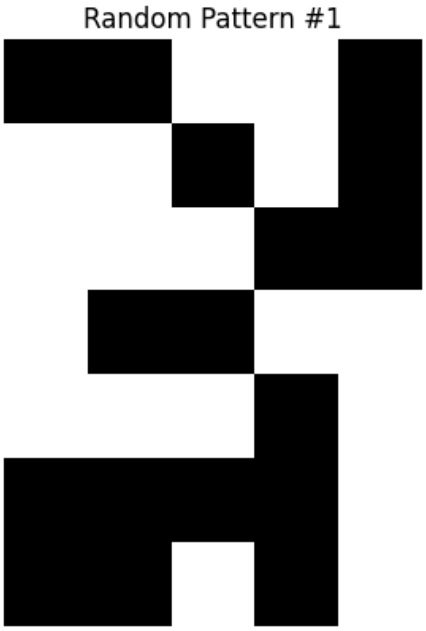


Figure 1

The best way to learn something is by doing it—this is especially true when it comes to understanding the subtle details that often get lost in theoretical explanations. This is even more helpful for those who may not be fond of long summation operations and mathematical derivations.

Let's now focus on the project itself. We'll work with three patterns laid out on a 7-row by 5-column grid. However, to process them in a Hopfield network, we need to convert these patterns into vectors by flattening the grid. This means taking the 7 rows and placing them one after another, resulting in vectors with 35 elements. Consequently, our Hopfield network will consist of 35 neurons.

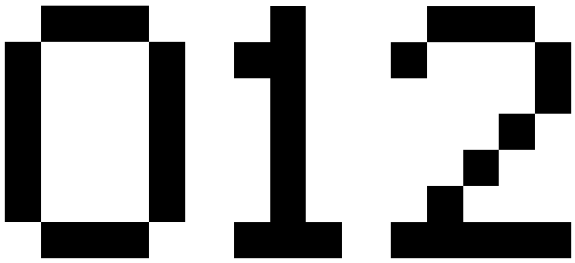
It's also important to note that we're using only three patterns for a specific reason: network stability. In this case, we are working with the digits 0, 1, and 2. Adding more digits could lead to instability in the network's convergence, reducing its effectiveness.

Object-Oriented Design Inspired by Machine Learning Libraries

To make our implementation more intuitive and modular, we adopt an **object-oriented approach**, directly inspired by the structure used in popular machine learning libraries in Python, such as `scikit-learn` and `PyTorch`. This design allows us to encapsulate the network's behavior and internal state within a class, making the code easier to maintain, reuse, and extend.

We define a class called `HopfieldNet` that contains all the core components of a Hopfield network:

```
import numpy as np
import random
```



(a) 0 (b) 1 (c) 2

Figure 2: Patterns

```python
import matplotlib.pyplot as plt
import matplotlib.animation as animation

class HopfieldNet:
    def __init__(self):
        self.W = None
        self.patterns = []

    def train(self, patterns):
        self.patterns = [np.array(p).flatten() for p in patterns]
        n = len(patterns[0])
        self.W = np.zeros((n, n))
        for p in self.patterns:
            p = p.reshape(n, 1)
            self.W += np.outer(p, p)
        np.fill_diagonal(self.W, 0)
        self.W /= len(patterns)

    def sign(self, x):
        return np.where(x >= 0, 1, -1)

    def energy(self, s):
        return -0.5 * s @ self.W @ s

    def recall(self, initial_pattern, max_iter=500):
        s = np.array(initial_pattern).flatten()
        for iter in range(max_iter):
            s_new = self.sign(self.W @ s)
            if np.array_equal(s_new, s):
                break
            s = s_new

        # Check match with trained patterns or their inverses
        for idx, p in enumerate(self.patterns):
            if np.array_equal(s, p) or np.array_equal(s, -p):
                return idx, s.tolist(), iter

        # No match found with any trained pattern or its inverse
        return -1, s.tolist(), iter

    def recall_with_history(self, initial_pattern, max_iter=500):
        s = np.array(initial_pattern).flatten()
```

```
        history = [s.copy()]

        for _ in range(max_iter):
            s_new = self.sign(self.W @ s)
            history.append(s_new.copy())
            if np.array_equal(s_new, s):
                break
            s = s_new

        for idx, p in enumerate(self.patterns):
            if np.array_equal(s, p) or np.array_equal(s, -p):
                return idx, history, len(history)-1

        return -1, history, len(history)-1
```

Class Description: `HopfieldNet`

- `__init__()`: Initializes the network. Sets the weight matrix `W` to `None` and prepares a list to store the flattened training patterns.
- `train(patterns)`: Trains the network using the Hebbian learning rule. Each input pattern is flattened (from 2D to 1D), and the weight matrix is computed via the outer product of each pattern with itself. The diagonal is zeroed out to remove self-connections, and the result is normalized by the number of patterns.
- `sign(x)`: Helper method that implements the bipolar activation function, assigning 1 to non-negative values and -1 to negative ones.
- `energy(s)`: Computes the **energy** of a given state vector `s`. This is useful for analyzing the dynamics of the network, as Hopfield networks are energy-minimizing systems.
- `recall(initial_pattern, max_iter=500)`: Attempts to recall a stored pattern from a noisy or incomplete input. The state is updated iteratively until convergence or until the maximum number of iterations is reached. If the final state matches (or is the inverse of) one of the trained patterns, it returns the index, the final state, and the number of iterations.
- `recall_with_history(initial_pattern, max_iter=500)`: Similar to `recall`, but stores the state at each iteration. This is particularly useful for visualization, debugging, or animation of the convergence process.

This structured design not only follows good programming practices but also helps users familiar with machine learning frameworks to understand the flow of the code at a glance.

External Utility Functions

```
def show_pattern(pattern, size=(7,5), title="Pattern"):
    matrix = np.array(pattern).reshape(size)
    plt.imshow(matrix, cmap='gray_r')
    plt.title(title)
    plt.axis('off')
    plt.show()

def add_noise(pattern, num_bits=5):
    noisy_pattern = pattern.copy()
    indices = random.sample(range(len(noisy_pattern)), num_bits)
    for i in indices:
        noisy_pattern[i] *= -1
    return noisy_pattern
def create_animation(history, size=(7, 5), title_prefix="Step", interval=500,
↪   save_as=None):
    fig, ax = plt.subplots()
```

```
    im = ax.imshow(np.array(history[0]).reshape(size), cmap='gray_r')
    ax.axis('off')

    def update(frame):
        im.set_array(np.array(history[frame]).reshape(size))
        ax.set_title(f"{title_prefix} {frame}")
        return [im]

    ani = animation.FuncAnimation(fig, update, frames=len(history), interval=interval,
    ↪  blit=True)

    if save_as:
        ani.save(save_as, writer='pillow')
    else:
        plt.show()
```

In addition to the `HopfieldNet` class, we implemented a set of external helper functions to assist with testing and visualizing how the network behaves during recall:

- `show_pattern(pattern, size, title)`: Displays a flattened pattern as a 2D image using `matplotlib`. This is useful for visual verification of patterns before and after recall.
- `add_noise(pattern, num_bits)`: Introduces noise to a pattern by flipping the sign of a specified number of random bits. This simulates corrupted or incomplete inputs to test the network's robustness.
- `create_animation(history, size, title_prefix, interval, save_as)`: Creates an animation that shows the evolution of the pattern over time as it converges to a stable state. This is particularly useful for understanding the dynamics of the network. Optionally, the animation can be saved as a GIF using `pillow`.

These functions were kept outside the class to preserve the network's abstraction while facilitating experimentation and visualization.

Implementation and Experimentation

With the class and helper functions defined, we proceed to implement and test our Hopfield network. The experiment consists of the following steps:

1. Define three training patterns, each represented as a vector of 35 bipolar values (± 1), corresponding to a 7×5 grid.
2. Train the Hopfield network with these patterns using the Hebbian rule.
3. Generate 30 random bipolar patterns to test the network's ability to converge to one of the trained patterns.
4. For each random input:
   - Display the original random pattern.
   - Feed it into the network for recall.
   - Display the resulting pattern returned by the network.
   - Store the index of the converged pattern for statistical analysis.
5. Finally, summarize the results by plotting the frequency with which the network converged to each trained pattern—or failed to converge to any.

```
# --- Setup network and patterns ---

# Training patterns
patterns = [

    ↪  [1,-1,-1,-1,1,-1,1,1,1,-1,-1,1,1,1,-1,-1,1,1,1,-1,-1,1,1,1,-1,-1,1,1,1,-1,1,-1,-1,-1,1],
```

```
    [1,1,-1,1,1,1,-1,-1,1,1,1,1,1,-1,1,1,1,1,1,-1,1,1,1,1,1,-1,1,1,1,1,-1,-1,-1,1],

    ↪  [1,-1,-1,-1,1,-1,1,1,1,1,-1,1,1,1,1,1,-1,1,1,1,1,-1,1,1,1,1,-1,1,1,1,1,-1,1,1,1,1,-1,-1,-1,-1,-1],
]

network = HopfieldNet()
network.train(patterns)

num_tests = 30
size = (7,5)  # Adjust according to your patterns
n = len(patterns[0])
results = []

for i in range(num_tests):
    # Create a totally random pattern (values 1 or -1)
    random_pattern = [random.choice([1, -1]) for _ in range(n)]

    show_pattern(random_pattern, title=f"Random Pattern #{i+1}")

    idx, recalled, iterations = network.recall(random_pattern)
    results.append(idx)

    show_pattern(recalled, title=f"Recalled Pattern #{i+1} (Index {idx})")

# Plot convergence frequency
import collections

count = collections.Counter(results)
indices = list(range(-1, len(patterns)))  # -1 means not recognized

frequencies = [count[i] if i in count else 0 for i in indices]

labels = ['Not recognized'] + [f'Pattern {i}' for i in range(len(patterns))]

plt.bar(labels, frequencies, color='skyblue')
plt.xticks(rotation=45)
plt.ylabel('Frequency')
plt.title('Convergence Frequency to Each Pattern')
plt.show()
```

This test is crucial for observing how the network behaves with inputs that are not only noisy but also completely unstructured. The convergence analysis gives insight into the attractor dynamics of the Hopfield network and helps assess the influence of the training patterns on its memory capacity.

The patterns used were intentionally selected to represent digits (0, 1, and 2) in a stylized binary form, with each digit encoded into a 7×5 grid. These grids were flattened to vectors of length 35 before being passed into the network.

A summary bar chart was generated to show how many times the network converged to each stored pattern, or failed to recognize any of them (marked as "Not recognized").

This experiment demonstrates both the capabilities and limitations of the Hopfield model when storing and recalling a small number of patterns in a low-dimensional setting.

Results and Observations

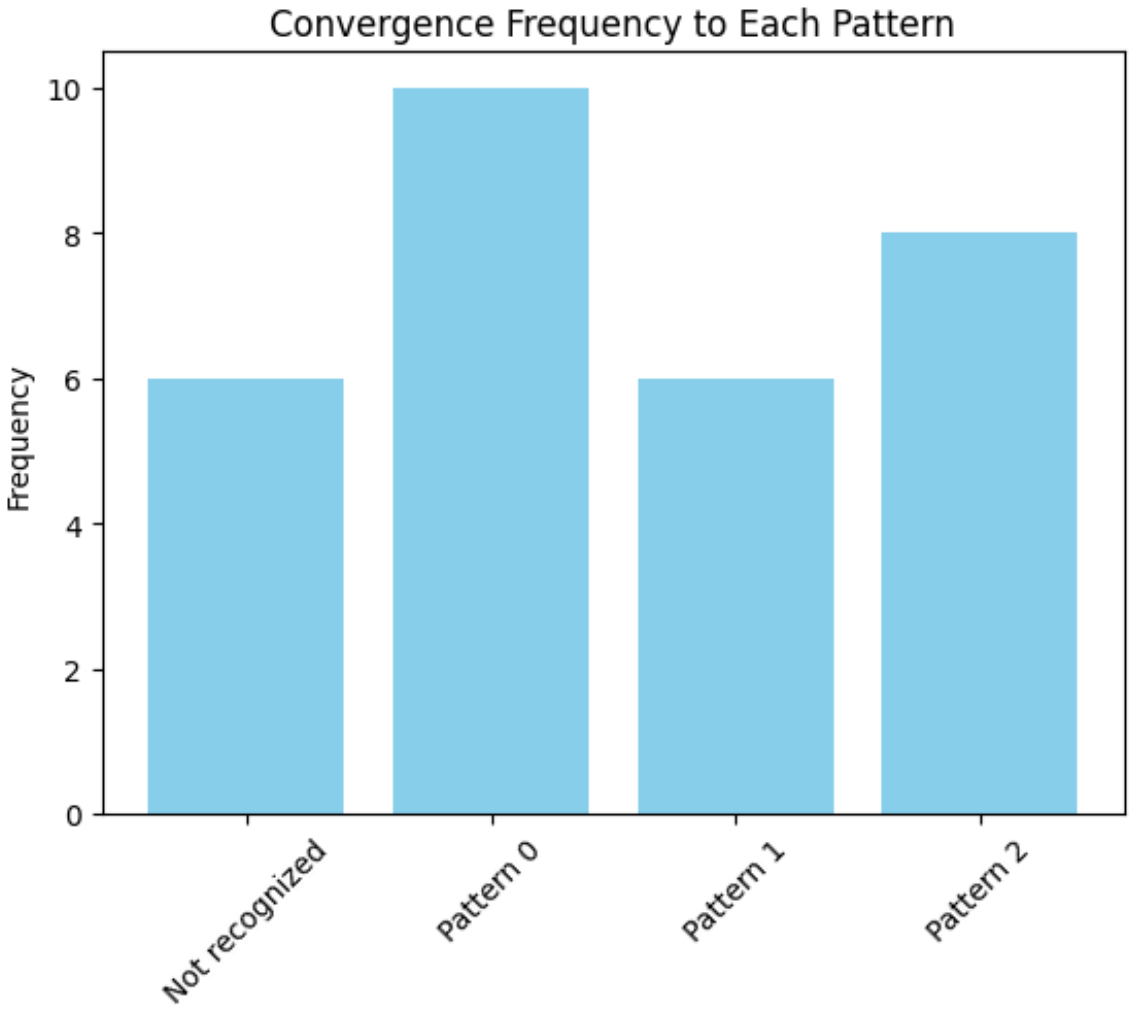
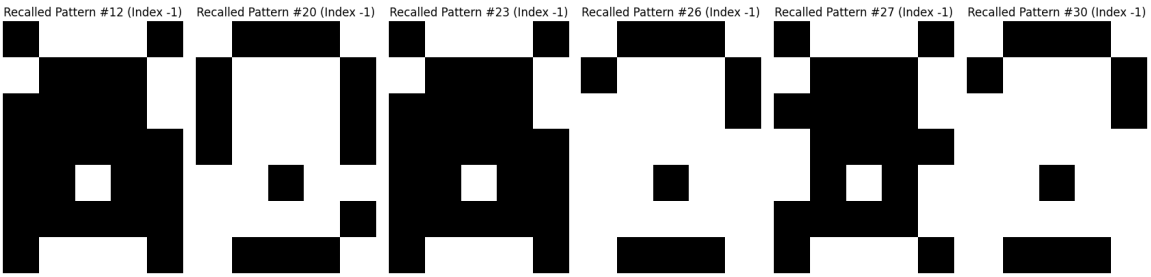


Figure 3

Out of the 30 random test patterns generated, the network successfully recognized and converged to one of the trained patterns (or their inverses) in 24 cases. This highlights the network's ability to not only retrieve original patterns but also correctly identify their negatives, which are equally valid stable states due to the symmetry of the Hopfield model.

However, in the remaining 6 cases, the network did not converge to any of the trained patterns or their inverses. Instead, it settled into non-recognizable configurations known as **spurious states**. These are stable states that emerge naturally from the dynamics of the network but do not correspond to any of the intended memories. Among these 6 failures, a total of 3 distinct spurious patterns were observed.



(a) (b) (c) (d) (e) (f)

Figure 4: Spurious Patterns

This behavior is expected in Hopfield networks, especially when storing multiple patterns in a relatively small network. It illustrates the inherent trade-off between capacity and reliability in associative memory systems.

References

[1] Mehlig, B. (2021). *Machine learning with neural networks*. Department of Physics, University of Gothenburg.